GRAPHER MODERNIZATION — MASTER AGENT IMPLEMENTATION ORDER

Directive

You are the implementation agent responsible for modernizing **Grapher**.

This is not a greenfield rewrite and it is not an invitation to replace Grapher with a different architecture because another design is more fashionable.

Your job is to:

1. inspect the actual Grapher repository;
2. understand how it currently works;
3. preserve working behavior;
4. generalize its data model beyond software development;
5. make lifecycle, truth, provenance, mission scope, and verification first-class concepts;
6. improve its retrieval behavior for autonomous agents;
7. preserve and upgrade its Cursor integration;
8. preserve and upgrade its deep-ingest behavior;
9. make its Dash visualization a first-class consumer of the same canonical model;
10. make Grapher suitable to become the knowledge substrate used by a future Agent Hub / Dreadnought system.

You are implementing **Grapher first** in a deliberate sequence:

GRAPER → AGENT HUB → AUDITOR

Therefore:

- Do not redesign Agent Hub in this task.
- Do not redesign Auditor in this task.
- Do not implement Dreadnought orchestration in this task.
- Do not add a hard runtime dependency on Agent Hub.
- Do not make Grapher responsible for agent scheduling.
- Do not make Grapher responsible for determining whether an actor is actually authorized to perform an action.
- Do make Grapher capable of recording the information those systems will later need.
- Do expose clean programmatic primitives that Agent Hub and Auditor can consume later.

The intended architecture is:

Grapher knows.
Agent Hub coordinates.

Auditor observes and verifies.
Dreadnought ultimately arbitrates.

Do not blur those responsibilities.

---

1. PRIMARY OBJECTIVE

Upgrade Grapher from a useful semantic knowledge store into a:

> **general-purpose, lifecycle-aware, provenance-aware, truth-aware work graph for humans and autonomous agents.**

It must represent not only what information exists, but also:

- what kind of information it is;
- what lifecycle stage it belongs to;
- whether it describes a proposal, current reality, canonical specification, history, rejection, or superseded state;
- whether a work item is active, blocked, complete, cancelled, etc.;
- whether a claim has been verified;
- what evidence supports it;
- who or what created it;
- in what agent/session/mission context it was created;
- which generation of a mission it belongs to;
- whether its provenance is trusted, merely declared, contested, or invalidated;
- what it supersedes;
- what it implements;
- what it contradicts;
- what it depends on;
- what current state should win during retrieval.

The graph must still preserve history.

Do not solve stale information by deleting history.

Do not solve ambiguity by pretending conflicting facts agree.

Do not solve autonomous-agent retrieval by simply returning the newest nodes.

---

2. CONTROL EVIDENCE YOU MUST READ BEFORE MODIFYING CODE

Before making implementation changes, inspect the repository and available Grapher artifacts.

At minimum identify and understand:

- CLI entry point;
- command parser;
- graph reader;
- graph writer;
- JSON schema or implicit schema;
- vector/embedding subsystem;
- search implementation;
- ranking implementation;
- relationship traversal;
- ingest implementation;
- pending ingest queue;
- Cursor installation logic;
- generated Cursor rules;
- generated Cursor skills;
- Dash application;
- Dash callbacks;
- Dash graph adapter;
- config reader;
- validation code;
- migration code, if any;
- backup behavior;
- tests;
- fixtures;
- README/help text.

You must also study existing graph outputs before deciding what "normal" data actually looks like.

Important existing Grapher artifacts include:

```
.grapher/
    config.json
    knowledge.json
    vectors.json
```

The case-study Grapher installation demonstrates existing generated Cursor integration resembling:

```
.cursor/rules/grapher.mdc
.cursor/skills/grapher-ingest/SKILL.md
```

The current integration already teaches agents that:

- Grapher is shared knowledge.
- Search should occur before acting.
- Agents should record durable discoveries.
- Images are not filename stubs.
- Video is not pathname metadata.
- Audio is not pathname metadata.
- Deep content understanding belongs in `content`.
- Superseded truth should not outrank current truth.
- Lifecycle stages exist.
- `.grapher/knowledge.json` acts as shared memory.

Do not regress any of this behavior.

---

3. EXECUTION DISCIPLINE

Do not begin by changing files.

Begin with a repository reconnaissance report.

Determine:

1. programming language;
2. package/module structure;
3. CLI framework;
4. persistence implementation;
5. graph representation in memory;
6. vector representation;
7. search path;
8. mutation path;
9. Cursor integration path;
10. Dash integration path;
11. existing test framework;
12. existing config conventions.

Then run the current tests.

If no tests exist for a component, establish a reproducible baseline manually before changing it.

Record:

- baseline test command;
- baseline test result;
- current Grapher version if one exists;
- current CLI help;
- current schema version;
- current default profile/domain assumptions;
- current command behavior;
- current Dash launch behavior;
- current ingest behavior.

Do not modify behavior before you can describe what behavior currently exists.

---

4. DO NOT REWRITE WITHOUT CAUSE

Preserve repository conventions wherever reasonable.

Do not:

- replace the CLI framework without necessity;
- replace JSON persistence merely because a database seems cleaner;
- replace Dash merely because another visualization framework exists;
- replace embeddings/vector logic without necessity;
- introduce a server architecture if the application is currently local;
- introduce an ORM;
- introduce an external service dependency;
- introduce network requirements for ordinary local Grapher use;
- introduce Agent Hub as a required dependency.

A surgical evolution is preferred over a ground-up rewrite.

If a structural rewrite is truly unavoidable, document:

1. why the existing architecture cannot satisfy the requirements;
2. what exact behavior would otherwise be impossible;
3. migration impact;
4. compatibility impact;
5. test evidence proving no required behavior was lost.

---

5. CANONICAL FILE OWNERSHIP

Grapher must have a clear ownership model.

The canonical semantic graph remains Grapher-owned.

Prefer the existing layout unless repository architecture requires otherwise:

```
.grapher/
    knowledge.json      # canonical durable graph
    vectors.json        # derived retrieval/index state
    config.json         # project Grapher configuration
```

Add an append-only mutation journal if an equivalent does not already exist:

```
.grapher/history.jsonl
```

or another repository-consistent name.

The distinction must be explicit:

knowledge.json

Canonical graph truth/history.

vectors.json

Derived search acceleration data.

It must not become an independent source of truth.

If vectors can be regenerated, they are a cache/index, not canonical state.

config.json

Project-level configuration.

history.jsonl

Append-only record of meaningful graph mutations.

The mutation journal is not a second graph.

It records how the canonical graph changed.

---

6. VECTOR INDEX RULES

Inspect how `vectors.json` is currently generated.

Do not blindly regenerate all embeddings merely because non-content metadata changes.

Determine whether vector identity depends on:

- node content only;
- title + content;
- tags;
- entire serialized node;
- node updated timestamp;
- another hash.

Refactor indexing if necessary so semantic metadata changes such as:

```
status
workflow_state
verification
mission generation
actor provenance
```

do not require an expensive re-embedding when title/content have not semantically changed.

If a content-bearing field changes, update the relevant vector.

If a node is deleted or merged, remove or invalidate its vector.

If the vector file is missing or corrupted, Grapher should be capable of rebuilding it from canonical graph data.

A corrupt vector cache must never corrupt `knowledge.json`.

---

7. GENERALIZE GRAPHER BEYOND SOFTWARE

The current tool must no longer implicitly assume that every graph represents source code.

A graph may represent:

- software;
- hardware;

- physical product design;
- research;
- marketing campaign;
- construction work;
- business processes;
- publishing;
- a museum exhibit;
- event production;
- organizational operations;
- product development;
- industrial design;
- documentation;
- creative work;
- investigations;
- maintenance operations;
- mixed projects.

The default profile must become:

```
general
```

not:

```
software
```

Software remains a first-class supported profile.

It simply stops being the universal assumption.

---

8. GRAPH KIND AND LIFECYCLE STAGE MUST BE ORTHOGONAL

These are different concepts.

Graph kind

Describes what relationships the graph is emphasizing.

Built-in kinds:

```
knowledge
brainstorm
concept
requirements
decision
design
dependency
roadmap
implementation
launch
operations
retrospective
```

Lifecycle stage

Describes where the represented work sits in its lifecycle.

Use these exact canonical stage values:

```
ideation
designing
planning
developing
launching
maintaining
```

Canonical order:

```
ideation
→ designing
→ planning
→ developing
→ launching
→ maintaining
```

A graph may have multiple kinds.

A graph may have multiple lifecycle stages.

A node may have one or more applicable lifecycle stages.

Do not hard-code mappings such as:

```
design graph = designing stage
```

That is false.

A design decision may be created during maintaining.

A dependency graph may span planning through launching.

An operations graph may contain design work for an operational improvement.

---

9. CLI SUPPORT FOR KINDS AND STAGES

Support repeat flags:

```
grapher init --kind design --kind decision
```

Support comma-separated values:

```
grapher init --kind design,decision
```

Do the same for stages:

```
grapher init --stage designing --stage planning
```

and:

```
grapher init --stage designing,planning
```

Add:

```
--kind
--stage
--all-stages
--domain
--profile
```

If an older command uses `--type` to describe graph kind, preserve it as an alias where practical and emit a deprecation notice.

Do not internally use the same variable or enum for:

```
graph kind
```

and:

```
node type
```

Normalize repeated and comma-separated inputs to unique ordered arrays.

Unknown built-in values should produce an error that:

1. identifies the unknown value;
2. lists canonical built-ins;
3. explains how custom values may be configured.

---

10. PROFILES

Ship at least:

```
general
software
product
research
campaign
operations
```

A profile may configure preferred:

- kinds;
- stages;
- node types;
- relations;
- evidence types;
- prompts;
- aliases;
- checkpoints;
- ranking hints.

Profiles must extend one common graph model.

Do not fork the schema by profile.

A software graph and a research graph must still be mutually understandable to Grapher.

---

11. SCHEMA VERSION 2

Implement schema version 2.

Preserve the recognizable structure of current graphs.

Conceptually:

```
{
  "version": 2,
  "graph": {},
  "nodes": {},
  "edges": []
}
```

Do not convert nodes from keyed objects to arrays unless the current codebase gives an overwhelming technical reason.

Existing convention:

```
"nodes": {
  "some-node": {
    "id": "some-node"
  }
}
```

must preserve the invariant:

```
node dictionary key == node.id
```

Migration validation must explicitly enforce this.

---

12. GRAPH METADATA

Add normalized top-level graph metadata.

Conceptually:

```
{
  "graph": {
    "name": "project-name",
    "domain": "general",
    "profile": "general",
    "kinds": ["knowledge"],
    "stages": [],
    "created_at": "...",
    "updated_at": "..."
  }
}
```

Use repository naming conventions where appropriate.

Do not manufacture a creation timestamp when the graph's real creation date cannot be established.

If migration needs a timestamp for a newly introduced metadata field, distinguish:

```
graph created
```

from:

```
schema migrated
```

where practical.

---

13. CORE NODE FIELDS

Preserve all current node fields, including:

```
id
type
title
content
path
tags
meta
created_at
updated_at
```

Add normalized support for:

```
stage
status
workflow_state
verification
evidence
source_refs
owners
started_at
completed_at
due_at
scope
provenance
finalized_at
```

Fields may remain optional on disk when backward compatibility benefits from it.

The reader must expose normalized defaults.

---

14. TRUTH STATUS

Truth status answers:

> How should a future reader treat this node as a representation of reality or specification?

Canonical statuses:

```
unclassified
proposed
current
canonical_spec
superseded
historical
rejected
deprecated
```

Meanings:

unclassified

Legacy or uncategorized information.

Do not assume it is current.

proposed

Suggested future direction or draft.

It is not current reality.

current

Represents present known reality within its scope.

canonical_spec

Authoritative specification, requirement, policy, design contract, or frozen reference.

A canonical specification can differ from current implementation.

That difference must remain representable.

superseded

Replaced by a newer node.

historical

Accurate historical context that is no longer current.

rejected

Explicitly considered and rejected.

deprecated

Should no longer be used, though preserved for history or compatibility.

Never infer truth status solely from timestamp recency.

---

15. WORKFLOW STATE

Workflow state answers:

> What is happening with the work represented by this node?

Canonical values:

```
not_started
active
blocked
completed
cancelled
on_hold
not_applicable
```

This is independent from truth status.

Example:

A task can be:

```
workflow_state = completed
```

while the resulting finding can be:

```
status = current
verification = verified
```

Do not encode all three meanings into one `status` field.

---

16. VERIFICATION STATE

Verification answers:

> How strongly has this claim, implementation, outcome, or result actually been checked?

Canonical values:

```
unverified
partially_verified
verified
failed
not_applicable
```

Do not infer `verified` merely because:

- an agent says "done";
- a task is completed;
- content sounds confident;
- a commit exists;
- a file exists.

Verification requires supporting evidence appropriate to the domain.

---

17. EVIDENCE

Support evidence records such as:

```
{
  "type": "test",
  "ref": "go test ./...",
  "summary": "All tests passed",
  "recorded_at": "..."
}
```

Built-in evidence categories should include at least:

```
test
file
document
image
measurement
observation
commit
conversation
external_source
command
log
other
```

Do not require software evidence for non-software projects.

Examples:

Software

- test results;

- files;
- commits;
- logs;
- benchmark measurements.

Physical product

- dimensions;
- drawings;
- photographs;
- test fit;
- prototype results.

Campaign

- approved creative;
- channel output;
- analytics;
- stakeholder approval.

Research

- dataset;
- paper;
- experiment;
- methodology;
- measurement.

Operations

- incident log;
- service metric;
- checklist;
- inspection.

Evidence may reference another Grapher node.

Prefer stable references.

Do not duplicate giant source documents inside evidence summaries.

---

18. MISSION / PROJECT / GENERATION SCOPE

This is a mandatory extension derived from multi-agent case-study evidence.

A fact must be able to state the operational scope in which it was created.

Add an optional normalized `scope` object.

Conceptually:

```
{
  "scope": {
    "workspace_id": "workspace-name",
    "project_id": "terminal",
    "mission_id": "ubuntu-prototype",
    "generation_id": "ubuntu-prototype-002"
  }
}
```

Names may differ to conform to repository conventions.

The important capability is not the exact property spelling.

Grapher must distinguish:

```
project
mission
mission generation
```

A reopened mission is **not** the same execution generation as the earlier mission.

A generic:

```
mission complete
```

statement from generation 1 must not silently satisfy generation 2.

Search and filtering must eventually support scope.

Suggested filters:

```
--project
--mission
--generation
```

Do not require these fields for ordinary Grapher use.

They are optional generalized scope primitives.

19. PROVENANCE

Every newly created semantic node must be capable of carrying provenance.

Conceptually:

```
{
  "provenance": {
    "actor_id": "codex-field",
    "actor_role": "field-agent",
    "session_id": "...",
    "source": "cursor",
    "recorded_at": "...",
    "integrity": "declared"
  }
}
```

The exact serialized layout may be improved during implementation.

The required concepts are:

```
actor identity
actor role
agent/session identity
source mechanism
recording time
provenance integrity
```

Canonical provenance integrity values:

```
unknown
declared
verified
contested
invalidated
```

Meaning:

unknown

Legacy or absent provenance.

declared

The creator supplied its identity/role, but Grapher itself has no external proof.

verified

An external trusted integration supplied authoritative identity evidence.

Grapher must not award this merely because an agent says it is verified.

contested

There is known reason to doubt authority or authorship.

invalidated

Preserved as evidence/history but explicitly excluded from authoritative interpretation.

This field is separate from truth status.

A node may contain factually useful information while having invalidated provenance.

---

20. GRAPHER IS NOT AN AUTHENTICATION SERVICE

Do not turn Grapher into Agent Hub prematurely.

Grapher records provenance.

It does not independently decide whether:

```
codex-field
```

really is the assigned field agent.

It may receive trusted provenance later from Agent Hub.

Design the interface so a future caller can provide:

```
actor_id
role
session_id
mission_id
```

```
generation_id
authority/attestation reference
```

without Grapher needing to understand Agent Hub internals.

If external attestation is supplied, preserve it as an opaque stable reference.

Do not invent a cryptographic signing system during this task unless Grapher already has one.

---

21. APPEND-ONLY MUTATION JOURNAL

Add an append-only semantic mutation journal unless the existing application already provides equivalent functionality.

Each semantic mutation should record enough information to reconstruct:

- what action occurred;
- when;
- affected graph/node/edge;
- actor context if supplied;
- mission/generation context if supplied;
- before/after hashes or equivalent stable change information;
- result;
- whether mutation succeeded;
- mutation source.

Possible actions:

```
node_created
node_updated
node_finalized
status_changed
edge_created
edge_removed
node_superseded
node_merged
checkpoint_refreshed
migration_started
migration_completed
migration_failed
curation_applied
```

Do not log every read.

Do not log cosmetic Dash interactions.

The journal must be append-only under ordinary commands.

It is for forensic reconstruction, not general semantic retrieval.

---

22. FINALIZED RECORDS

Some graph nodes represent durable historical actions:

- handoffs;
- acceptance decisions;
- audit completion;
- test results;
- incident records;
- releases;
- official checkpoints.

Support the concept of finalization.

A node with:

```
finalized_at
```

must not be casually rewritten.

After finalization, normal `add` / `update` behavior should reject destructive semantic replacement of the record.

Corrections should normally occur through:

```
new correcting node
+
supersedes / corrects relation
```

rather than silently rewriting history.

An explicit force mechanism may exist for administrative recovery, but:

- it must be difficult to invoke accidentally;
- it must produce a journal entry;
- it must not masquerade as normal editing.

23. MULTI-LAYER COMPLETION MUST BE REPRESENTABLE

Do not model all completion as a single boolean.

The control case demonstrated distinct states such as:

```
field work ready
arbiter acceptance
arbiter clock-out
audit completion
overall product scope completion
```

Grapher must be capable of preserving these as separate facts.

Do not hard-code those exact Dreadnought states into a universal workflow enum.

Instead support appropriate node types and precise relationships so these states can be modeled independently.

Add general-purpose node types where useful:

```
mission
session
handoff
acceptance
audit_record
event
claim
```

Names may be adapted to the actual type registry.

Example conceptual graph:

```
field-handoff
    applies_to → mission-generation-002

field-handoff
    authored_by → codex-session-abc

arbiter-acceptance
```

```
    accepts → mission-generation-002

audit-record
    audits → mission-generation-002
```

This allows later systems to ask:

> Is the field agent finished?

without confusing that question with:

> Has the arbiter accepted the work?

---

24. EXPAND GENERAL NODE TYPES

Preserve current node types.

Add configurable built-ins including:

```
idea
question
concept
goal
requirement
constraint
assumption
option
decision
component
artifact
document
image
video
audio
person
organization
stakeholder
task
milestone
deliverable
dependency
risk
issue
incident
```

```
finding
metric
policy
instruction
checkpoint
mission
session
handoff
acceptance
audit_record
event
claim
```

Do not force users to choose only from built-ins.

Support configured custom types.

Prompts should choose semantically appropriate types.

Do not label everything a `finding`.

---

25. RELATION REGISTRY

Preserve current relations.

Keep:

```
related
```

as a legal fallback.

Treat it as low-information.

Add a relation registry.

Required relations should include:

```
references
depicts
applies_to
related
```

```
supersedes
implements
deviates_from
fixes
caused_by

depends_on
blocks
enables
precedes
follows

verified_by
evidenced_by
derived_from

decided_by
chosen_over

satisfies
violates
constrains

owns
assigned_to
authored_by
performed_by
observed_by

produces
part_of
affects

hands_off
accepts
audits

launches
maintains
```

Relation definitions should contain where useful:

```
canonical name
human description
directionality
inverse label
```

```
source guidance
target guidance
```

Do not prevent legitimate unconventional relations solely because source/target types are unusual.

Guidance is not the same thing as rigid ontology enforcement.

---

26. SUPERSESSION

Canonical direction:

```
NEW NODE --supersedes--> OLD NODE
```

Not the reverse.

When creating such an edge, Grapher may propose:

```
OLD.status = superseded
```

but do not silently perform semantic status changes in non-interactive operation unless explicitly requested.

Support equivalent behavior to:

```
grapher curate supersede NEW OLD
```

and an explicit status-effect flag.

Supersession chains must be traversable.

Detect:

- cycles;
- self-supersession;
- multiple competing current replacements;
- dangling supersession references;
- current nodes that are themselves superseded;
- superseded nodes without an identifiable replacement.

---

27. CONTRADICTIONS

Do not flatten contradictory nodes.

Grapher must preserve disagreement.

Support either a built-in relation such as:

```
contradicts
```

or an equivalent explicit contradiction representation.

A contradiction can occur because of:

- scope differences;
- lifecycle changes;
- different mission generations;
- canonical spec vs implementation;
- genuinely unresolved disagreement.

Search must distinguish:

```
resolved by supersession
```

from:

```
still unresolved
```

A later timestamp is not proof of resolution.

---

28. RETRIEVAL MUST BECOME TRUTH-AWARE

Semantic similarity alone is insufficient.

Search ranking should consider:

```
semantic similarity
truth status
verification
scope
```

```
mission generation
graph kind
lifecycle stage
edge context
checkpoint relevance
query intent
recency
provenance integrity
```

Do not make recency dominant.

Do not make provenance dominant for every query.

For an authoritative-current-state query, contested or invalidated provenance should strongly reduce authority.

For a historical incident query, an invalidated acceptance node may still be relevant as incident evidence.

Ranking must therefore be contextual.

---

29. DEFAULT TRUTH PREFERENCE

For ordinary current-state queries, prefer approximately:

```
current
canonical_spec
current checkpoint
proposed
historical
unclassified
superseded
deprecated
rejected
```

This is not an absolute fixed score.

Examples:

"What was the original requirement?"

Prefer:

```
canonical_spec
```

even if implementation has changed.

"What does the system do now?"

Prefer:

```
current + verified implementation
```

"Why did the failure happen?"

Prefer:

```
incident
historical finding
root cause
verification
```

"What are we planning next?"

Prefer:

```
proposed
active roadmap
```

---

30. GENERATION-AWARE RETRIEVAL

If mission generation is known, prefer nodes from that generation when answering generation-specific questions.

Example:

```
ubuntu-prototype generation 1 = remediation mission
ubuntu-prototype generation 2 = reopened full prototype mission
```

Generation 1 completion must not automatically be treated as generation 2 completion.

If a search spans generations, clearly identify the generation of each state-bearing result.

If the graph contains:

```
generation 1 completed
generation 2 active
```

a generic current mission-state query should not return:

```
completed
```

without qualification.

---

31. PROVENANCE-AWARE RETRIEVAL

When authoritative status matters:

Prefer:

```
verified provenance
```

then:

```
declared provenance
```

then:

```
unknown provenance
```

Treat:

```
contested
invalidated
```

as subordinate for authoritative-current-state answers.

But do not hide them.

They may be essential historical evidence.

Search output should be capable of saying conceptually:

```
A later acceptance record exists, but its provenance is invalidated.
The latest uncontested authoritative state remains field-ready, awaiting arbiter
acceptance.
```

That capability is a major design goal.

---

32. SEARCH FILTERS

Add or preserve equivalents of:

```
--kind
--stage
--status
--workflow-state
--verification
--type
--tag
--project
--mission
--generation
--actor
--role
--as-of
--include-history
--include-superseded
--explain-ranking
```

Use the repository's established command syntax.

Do not invent an incompatible CLI hierarchy merely to match this document literally.

---

33. EXPLAINABLE RANKING

`--explain-ranking` must provide useful scoring detail.

A developer should be able to inspect factors such as:

```
semantic score
status adjustment
verification adjustment
stage match
kind match
scope match
mission-generation match
provenance adjustment
recency component
edge-context component
checkpoint component
final score
```

Do not expose meaningless internal floating-point noise without interpretation.

The point is to answer:

> Why did this result beat that result?

---

34. CONTEXT RETRIEVAL FOR AGENTS

When Cursor or another agent asks Grapher for context before work, return a bounded useful context packet.

Prefer:

1. current state;
2. canonical constraints;
3. relevant active decisions;
4. implementation evidence;
5. known contradictions;
6. blockers;
7. unresolved questions;
8. directly relevant history.

Do not dump every neighbor.

Do not bury the current answer under hundreds of historical findings.

---

35. CHECKPOINTS

Keep or implement a first-class:

```
checkpoint
```

node.

Checkpoint purpose:

> compact the current winning state of a bounded topic without deleting the underlying history.

Suggested operations:

```
grapher checkpoint create
grapher checkpoint refresh
grapher checkpoint list
```

Exact CLI hierarchy may follow repository conventions.

A checkpoint should link to its evidence.

Use relations such as:

```
derived_from
verified_by
implements
supersedes
```

A checkpoint should not become an untraceable AI summary.

---

36. CHECKPOINT REFRESH

Refreshing a checkpoint must:

1. identify its scope/topic;
2. retrieve current/canonical nodes;
3. follow supersession chains;
4. consider verification;
5. consider provenance integrity;
6. respect mission generation when scoped;
7. surface unresolved contradictions;
8. preserve historical evidence links;
9. produce a preview;

10. show a diff against the old checkpoint;
11. require confirmation before replacement unless explicitly auto-approved.

Do not overwrite conflicting facts into fake consensus.

---

37. CHECKPOINT STALENESS

A checkpoint must become stale when important supporting state changes.

Examples:

- supporting current node superseded;
- new contradiction created;
- verification failed;
- mission generation changed;
- canonical spec updated;
- relevant node invalidated;
- supporting node provenance becomes contested.

Audit should report stale checkpoints.

Search should preferably warn when a stale checkpoint is returned.

---

38. COMPACTION

Long-running graphs accumulate sediment.

Add review-first compaction.

Equivalent capability:

```
grapher curate compact --topic "<query>" --dry-run
```

Compaction may propose:

- consolidated current-state node;
- checkpoint;
- historical reclassification;
- supersession links;
- derived-from links.

Original nodes must remain unless an explicit destructive operation exists and is deliberately invoked.

Default compaction is non-destructive.

Preview must show:

```
nodes included
nodes excluded
why they were included/excluded
proposed consolidated content
proposed status changes
proposed relationships
contradictions
generation boundaries
provenance concerns
```

Do not merge conflicting mission generations as if they were one uninterrupted event.

---

39. VALIDATION

Implement or expand:

```
grapher validate
```

Structural validation should include:

- valid JSON;
- supported schema version;
- node key == node.id;
- unique node IDs;
- valid edge endpoints;
- valid relation names;
- valid configured custom relations;
- valid node types;
- valid custom node types;
- valid stages;
- valid truth statuses;
- valid workflow states;
- valid verification states;
- valid timestamps;
- evidence structure;
- scope structure;
- provenance structure;
- impossible finalization data;
- malformed source references;

- duplicate edges;
- self-invalid relations where applicable;
- supersession cycles.

Validation must not mutate.

---

40. AUDIT

`grapher audit` is semantic inspection.

It must not mutate.

Report at least:

```
node counts by type
node counts by stage
node counts by status
node counts by workflow state
node counts by verification
relation counts
percentage related edges
isolated nodes
weakly connected nodes
supersession health
supersession cycles
contradictions
verified nodes without evidence
finalized records modified incorrectly
stale checkpoints
scope/generation ambiguity
provenance unknown/contested/invalidated counts
possible duplicate breadcrumb clusters
low-quality ingest content
missing source references where expected
```

Add useful severity levels.

For example:

```
INFO
WARNING
```

```
ERROR
CRITICAL
```

Do not label ordinary legacy unclassified nodes as catastrophic errors.

---

41. CASE-STUDY-SPECIFIC AUDIT CONDITIONS

Use the lessons from Case Study 001 as generalized acceptance cases.

The graph must be capable of detecting or clearly representing situations equivalent to:

Premature completion

An older mission generation says complete while a later reopened generation exists.

Stale report

A report describes an earlier scope but remains superficially current.

Mixed provenance

A document contains records from different actors/phases.

Contaminated acceptance

An acceptance record exists, but provenance is contested or invalidated.

Role boundary interference

An actor records a decision normally belonging to another role.

Grapher cannot decide authorization on its own, but it must preserve enough provenance for a later Auditor to detect this.

Multiple completion layers

Field completion is not automatically arbiter acceptance.

Arbiter acceptance is not automatically audit completion.

---

42. CURATION COMMANDS

Implement or preserve capabilities equivalent to:

```
grapher curate status <node> <status>

grapher curate relate <from> <rel> <to>

grapher curate supersede <new> <old>

grapher curate merge <target> <source>

grapher curate compact --topic "<query>"
```

Consider additional provenance curation where appropriate:

```
grapher curate provenance <node> ...
```

and/or:

```
grapher curate invalidate <node> --reason "..."
```

Do not invent command proliferation if a cleaner existing hierarchy can implement the same functions.

Semantic curation must create mutation-history entries.

---

43. MIGRATION FROM VERSION 1

Implement safe migration equivalent to:

```
grapher migrate <path> --to 2
```

Support:

```
--dry-run
--infer
```

```
--yes
--no-backup
```

or repository-consistent equivalents.

Migration must be:

```
lossless
idempotent
atomic
auditable
```

---

44. MIGRATION MUST PRESERVE V1 SEMANTICS EXACTLY

For all original v1 nodes preserve:

```
id
dictionary key
type
title
content
path
tags
meta
created_at
updated_at
```

For all original edges preserve:

```
from
to
rel
note
created_at
```

Do not rewrite prose.

Do not normalize spelling inside content.

Do not reorder semantic content.

Do not silently replace old `related` relationships with guessed precise relations.

Migration is not curation.

---

45. MIGRATION BACKUPS AND ATOMICITY

Before destructive migration:

1. create a backup unless explicitly disabled;
2. write migrated state to temporary storage;
3. validate the temporary graph;
4. fsync where appropriate;
5. atomically replace canonical graph;
6. record migration result;
7. leave original intact if validation or write fails.

A failed migration must not leave half-written JSON.

A failed vector rebuild must not destroy the migrated canonical graph.

---

46. MIGRATION INFERENCE

`--infer` may propose safe semantic improvements.

Safe candidates include:

- explicit `SUPERSEDED by <id>` content;
- existing `superseded` tags;
- documents explicitly described as canonical;
- findings containing explicit successful test evidence.

Inference must be conservative.

Never infer:

```
current
```

because a node is newest.

Never infer:

```
verified
```

because an agent used confident language.

Never infer:

```
verified provenance
```

from self-declared identity.

Dry-run output must show every proposed semantic mutation.

Semantic inference should require explicit application.

---

47. DEEP DIRECTORY INGEST MUST BE PRESERVED

The current Grapher agent integration correctly treats media as first-class knowledge.

This must remain non-negotiable.

When ingesting a directory:

```
grapher ingest <dir>
```

the system may queue files, but ingestion is not semantically complete until the enriching agent has actually interpreted them.

A path is not knowledge.

---

48. IMAGE INGEST

For an image node, useful `content` may include:

- visible subjects;
- composition;
- UI layout;
- labels;
- text;
- diagrams;

- controls;
- visual relationships;
- colors where meaningful;
- relevant measurements when visible;
- project significance.

Unacceptable:

```
Image file at docs/foo.png
```

Acceptable:

```
Screen depicts the sampler home menu with MIC selected...
```

Do not regress this existing principle.

---

49. VIDEO INGEST

Video content should capture, where tools permit:

- scenes;
- meaningful timestamps;
- actions;
- screen state changes;
- spoken information;
- on-screen text;
- interaction sequence;
- project relevance.

If the enriching agent cannot actually inspect the audiovisual content, the node must state the limitation.

Never pretend a filename means the file was understood.

---

50. AUDIO INGEST

Audio content should capture:

- speech gist/transcript where applicable;
- music characteristics where relevant;
- sound effects;
- timing;

- audible defects;
- environmental context;
- project relevance.

Again:

A pathname is not semantic ingestion.

---

51. PENDING QUEUE SEMANTICS

Preserve the useful existing pattern:

```
ingest
→ pending queue
→ agent enrichment
→ add/update graph
→ scan
→ pending 0
```

A directory ingest cannot report semantic completion while required files remain path-only pending stubs.

If the user explicitly aborts ingestion, report pending items honestly.

---

52. GRAPH-WORTHY KNOWLEDGE THRESHOLD

Autonomous agents must not turn Grapher into an edit log.

Record graph-worthy information such as:

- architecture;
- canonical requirements;
- meaningful design choices;
- user-visible behavior;
- durable constraints;
- root causes;
- significant defects;
- verified fixes;
- important implementation state;
- operational procedure;
- launch requirements;
- reusable lessons;
- incidents;

- handoffs;
- acceptance decisions;
- significant deviations.

Generally do not create independent semantic nodes merely for:

- formatting edits;
- typo fixes;
- transient shell chatter;
- repeated statements;
- every file save;
- every command;
- every microscopic code modification.

The mutation journal and semantic graph serve different purposes.

---

53. CURSOR INTEGRATION — BEFORE WORK

Update the installed Cursor rules/skills so an agent begins by understanding the project graph.

The loop should be conceptually:

```
grapher search "<task context>"
grapher get <important-id>
grapher neighbors <important-id>
grapher audit
```

Commands must match actual implemented CLI.

Before acting, Cursor should identify:

```
profile
domain
graph kinds
lifecycle stages
project scope
mission/generation scope when supplied
current state
canonical constraints
open contradictions
superseded material
relevant evidence
```

---

54. CURSOR INTEGRATION — DURING WORK

Cursor should graph durable discoveries as work proceeds.

Examples:

- discovered requirement;
- root cause;
- intentional deviation;
- significant implementation decision;
- rejected alternative;
- blocker;
- test result;
- important failure;
- mission handoff.

Prefer updating a continuing current-state node when appropriate rather than creating hundreds of tiny duplicate findings.

Do not destroy old semantic truth when meaning has actually changed.

Use supersession.

---

55. CURSOR INTEGRATION — AFTER WORK

After significant work:

1. record changed current state;
2. record important evidence;
3. set verification honestly;
4. update workflow state;
5. create supersession links where necessary;
6. include supplied mission/generation context;
7. refresh or stale relevant checkpoints;
8. run validation;
9. report unresolved contradictions.

Do not claim graph completion merely because code work finished.

---

56. CURSOR PROVENANCE ENVIRONMENT

Design Cursor-facing Grapher commands so a calling environment can provide provenance without requiring Agent Hub.

Accept opaque metadata through an appropriate method such as:

- CLI options;
- environment variables;
- config/session context;
- programmatic API.

Likely concepts:

```
GRAPHER_ACTOR_ID
GRAPHER_ACTOR_ROLE
GRAPHER_SESSION_ID
GRAPHER_PROJECT_ID
GRAPHER_MISSION_ID
GRAPHER_GENERATION_ID
```

These names are illustrative.

Use repository-consistent naming.

Avoid requiring agents to manually repeat these flags on every command when a session context can safely provide them.

Explicit command arguments should override environment defaults.

Document precedence.

---

57. DASH IS FIRST-CLASS

Dash is not an unrelated viewer.

The architecture boundary must be:

Grapher owns

- schema;
- normalized model;
- validation;

- migration;
- search;
- ranking;
- relations;
- mutation;
- canonical persistence.

Dash owns

- visualization;
- interaction;
- filtering interface;
- detail presentation;
- view export.

Dash must consume Grapher's normalized model.

Do not reproduce a second schema inside callbacks.

---

58. SHARED NORMALIZATION LAYER

Create or identify one normalization/query layer used by:

- CLI;
- Cursor retrieval;
- Dash.

Do not duplicate enums such as:

```
statuses
stages
relations
verification values
```

inside Dash.

Dash should import/use the same registries.

---

59. DASH MUST OPEN V1 AND V2

Dash must be able to open a v1 graph through the normalized Grapher reader.

Opening a v1 graph must not silently migrate it.

Display a non-blocking notice that richer status/lifecycle/provenance views require migration/curation.

Dash exploration remains read-only by default.

---

60. DASH VIEW MODES

Implement/selectable modes including:

Knowledge Network

General semantic graph.

Lifecycle Flow

Arrange around:

```
IDEATION
DESIGNING
PLANNING
DEVELOPING
LAUNCHING
MAINTAINING
```

Unassigned legacy nodes remain visible in an appropriate unassigned group.

Dependency Graph

Emphasize:

```
depends_on
blocks
enables
precedes
follows
```

Detect cycles.

Do not render a fake DAG when the graph contains cycles.

Decision Graph

Emphasize:

```
goal
option
decision
constraint
evidence
chosen_over
supersedes
deviates_from
```

Roadmap

Emphasize:

```
milestones
deliverables
stages
owners
workflow
dates
blockers
dependencies
```

Do not invent dates.

Current State

Prefer:

```
current
canonical_spec
checkpoints
supporting evidence
```

Collapse history by default, but never delete it.

History / Supersession

Trace:

```
supersedes
deviates_from
fixes
caused_by
verified_by
```

Operations

Emphasize:

```
incidents
metrics
risks
policies
maintenance
verification
```

Provenance / Mission History

Add a view suitable for multi-agent graphs.

Emphasize:

```
mission
generation
session
handoff
acceptance
audit
actor provenance
contested/invalidated records
```

This view will be especially useful when Agent Hub and Auditor are upgraded later.

---

61. DASH FILTERS

Provide controls for:

```
view mode
graph kind
```

```
stage
node type
truth status
workflow state
verification
relation
tag
free-text
project
mission
generation
actor
role
provenance integrity
current only
include history
include superseded
show related edges
neighborhood depth
layout
reset
fit viewport
```

Hide advanced mission/provenance filters gracefully when the graph has no such data.

Do not clutter basic graphs unnecessarily.

---

62. DASH VISUAL SEMANTICS

Visual encoding should distinguish:

- node type;
- lifecycle stage;
- truth status;
- workflow state;
- verification;
- provenance integrity;
- relation type.

Do not communicate essential meaning using color alone.

Use combinations of:

- shape;
- border;

- line style;
- icon;
- badge;
- text;
- legend;
- tooltip.

Current/canonical nodes should be prominent.

Proposed nodes must not visually look completed.

Historical/superseded nodes should be subordinate but readable.

Contested/invalidated provenance must be clearly identifiable.

Failed verification must be obvious.

Blockers must be obvious.

---

63. DASH NODE DETAIL

Selecting a node should show:

```
title
ID
node type
full content
kinds
stages
truth status
workflow state
verification
tags
owners
path
source refs
evidence
scope
project
mission
generation
actor
role
session
```

```
provenance integrity
created/updated timestamps
started/due/completed
finalized_at
incoming relations
outgoing relations
supersession chain
checkpoint membership
checkpoint staleness
ranking explanation when relevant
```

Distinguish stored properties from display-only derived values.

---

64. DASH SUMMARY PANEL

Show useful graph health including:

```
visible nodes vs total
visible edges vs total
counts by type
counts by stage
counts by truth status
counts by verification
related-edge percentage
contradictions
supersession warnings
stale checkpoints
weakly connected nodes
provenance warnings
active mission generations
active filters
```

---

65. DASH PERFORMANCE

Graphs containing hundreds or thousands of nodes must remain usable.

Do not regenerate everything for cosmetic-only interactions.

Use the application's existing caching model where possible.

Consider:

- cached normalization;
- cached layouts;
- debounced search;
- neighborhood filtering;
- low-zoom label reduction;
- detail on hover/selection.

Do not silently omit nodes for performance.

If a rendering limit exists, disclose it.

---

66. DASH EDITING

Read-only is the default.

If Dash already supports edits:

All mutations must go through Grapher's canonical mutation service.

Never let callbacks ad hoc rewrite JSON.

Semantic changes such as:

```
status update
supersession
provenance invalidation
merge
checkpoint update
```

should show a preview and explicit confirmation.

Validation failure must result in no partial canonical write.

---

67. DASH EXPORT

Support the export formats the installed stack can reliably produce.

Prefer:

```
interactive HTML
PNG
SVG
filtered JSON
node CSV
edge CSV
```

Filtered JSON must be explicitly identified as:

```
exported view/subgraph
```

not the canonical graph.

Exports should contain useful metadata:

```
graph name
schema version
generated time
active filters
selected view
legend
```

Exporting must never mutate `knowledge.json`.

---

68. CONFIGURATION

Extend the current `.grapher/config.json` rather than inventing competing configuration.

Support project configuration for:

```
graph name
domain
profile
default kinds
default stages
custom node types
custom relations
status-ranking weights
stage aliases
relation aliases
checkpoint definitions
```

```
prompt guidance
include paths
exclude paths
Cursor integration behavior
graph-worthy threshold
provenance/session defaults where appropriate
```

Built-ins should work without a config.

Config extends defaults.

Users should not need to duplicate all built-in registries.

---

69. ALIASES

Support reasonable stage aliases:

```
design -> designing
development -> developing
launch -> launching
maintenance -> maintaining
```

Always serialize canonical stage values.

Relation aliases may also be configured.

Do not silently serialize alias spellings into canonical graph data.

---

70. TEST STRATEGY

This upgrade requires serious automated coverage.

Do not rely entirely on manual testing.

Add tests at the appropriate:

```
unit
integration
CLI
Dash/component
```

```
migration
fixture
```

levels.

---

71. V1 COMPATIBILITY TESTS

Test:

- v1 load;
- normalized defaults;
- node dictionary structure;
- edge preservation;
- search;
- Dash rendering;
- ingest compatibility.

A v1 graph must remain usable without migration.

---

72. MIGRATION TESTS

Test:

```
v1 → v2 losslessness
migration idempotence
backup creation
--no-backup
dry run
atomic failure
validation failure
inference preview
explicit inference apply
vector invalidation/rebuild
```

---

73. SCHEMA TESTS

Test:

- status;

- workflow state;
- verification;
- evidence;
- scope;
- provenance;
- finalization;
- custom node types;
- custom relations;
- aliases.

---

74. RELATION TESTS

Test:

```
supersedes direction
cycle detection
duplicate edges
missing endpoint
contradiction handling
custom relations
related fallback
```

---

75. SEARCH TESTS

Test queries where:

Case A

Current verified result should outrank older superseded implementation.

Case B

Canonical specification should outrank implementation when asking for original spec.

Case C

Historical incident should appear for root-cause question.

Case D

Proposed roadmap should appear for future-plan question.

Case E

Generation 2 active state should prevent generation 1 completion from being returned as present completion.

Case F

Invalidated acceptance should not outrank uncontested field handoff for authoritative closure status.

Case G

Invalidated acceptance should still appear when querying the interference incident.

These are critical.

---

76. CHECKPOINT TESTS

Test:

- creation;
- scope;
- refresh preview;
- diff;
- contradiction protection;
- supersession handling;
- staleness;
- mission-generation sensitivity;
- provenance sensitivity.

---

77. AUDIT TESTS

Prove audit is non-mutating.

Test detection of:

```
supersession cycles
dangling edges
verified without evidence
checkpoint staleness
generation ambiguity
contested provenance
invalidated authoritative records
```

```
related-edge overuse
isolated nodes
```

---

78. CURSOR INTEGRATION TESTS

Test generated Cursor installation content.

Confirm rules retain deep-media requirements.

Confirm rules describe:

```
search before acting
truth-aware retrieval
lifecycle
verification
supersession
mission/generation scope
provenance
graph-worthy threshold
after-work graph update
```

Do not leave old generated rules contradicting the new Grapher behavior.

---

79. DASH TESTS

Test at least:

- v1 adapter;
- v2 adapter;
- lifecycle ordering;
- filter combinations;
- reset;
- current/superseded visibility;
- provenance visibility;
- generation filters;
- node detail data;
- relation style registry;
- search selection;
- export metadata;
- canonical non-mutation.

---

80. NON-SOFTWARE FIXTURE

Create a small fixture proving Grapher is general.

Example:

```
museum exhibit
```

It might contain:

- idea;
- visitor goal;
- physical constraint;
- design option;
- decision;
- supplier;
- milestone;
- installation task;
- launch;
- maintenance procedure.

Span multiple lifecycle stages.

Do not include code terminology unless naturally necessary.

---

81. CASSIO FIXTURE

Continue using `cassio-brain.json` as the major real-world migration fixture.

Preserve all original semantic data.

Prove Grapher can represent:

- frozen canonical manual;
- current implementation;
- intentional implementation deviations;
- superseded implementation findings;
- root-cause incidents;
- verification;
- design evolution;
- current checkpoints;
- history.

Do not modify the fixture merely to make tests easier.

---

82. DREADNOUGHT CASE-STUDY FIXTURE

Create a compact test fixture derived from the structural lessons of the control case.

Do not needlessly import megabytes of case-study data.

Represent enough to test:

```
mission generation 1
field handoff 1
partial closure
mission reopened
mission generation 2
field handoff 2
stale earlier report
authentic arbiter missing
contaminated acceptance
invalidated provenance
```

Expected current interpretation:

```
technical field work ready/completed
authentic arbiter acceptance not established
```

The graph must be able to reach that conclusion without deleting the contaminated record.

This fixture will be highly valuable for later Agent Hub and Auditor development.

---

83. MUTATION JOURNAL TESTS

Test that semantic mutations create journal entries.

Test:

- node create;
- status change;
- supersession;
- merge;

- finalization;
- provenance invalidation;
- migration.

Test that reads do not spam the journal.

Test journal append failure behavior.

Do not permit journal failure to silently produce unknowable canonical mutation behavior.

Define and document the chosen transaction semantics.

---

84. FINALIZED RECORD TESTS

Test:

1. create node;
2. finalize node;
3. attempt ordinary semantic rewrite;
4. confirm rejection;
5. create correcting node;
6. supersede old record;
7. confirm history preserved.

---

85. DOCUMENTATION

Update README and CLI help.

Explain, in this order:

1. what Grapher is;
2. how to initialize it;
3. general/non-software use;
4. graph kind vs lifecycle stage;
5. truth status;
6. workflow state;
7. verification;
8. evidence;
9. relations;
10. search;
11. checkpoints;
12. provenance;
13. mission/generation scope;

14. audit;
15. migration;
16. compaction;
17. Cursor integration;
18. Dash;
19. software profile examples.

Do not make software the first explanatory example.

---

86. DOCUMENT A COMPLETE GENERAL EXAMPLE

Include something conceptually like:

```
grapher init museum-exhibit
  --profile product
  --kind brainstorm,design,roadmap
  --stage ideation,designing,planning

grapher add ...

grapher relate ...

grapher checkpoint create ...

grapher audit
```

Use the actual implemented command syntax.

Do not document commands that do not exist.

---

87. DOCUMENT CASSIO MIGRATION

Include actual equivalent commands for:

```
grapher migrate cassio-brain.json --to 2 --dry-run --infer

grapher migrate cassio-brain.json --to 2

grapher audit cassio-brain.json
```

Again, adapt to actual CLI structure.

88. DOCUMENT MULTI-AGENT CONTEXT

Document how an external agent system may supply:

```
actor ID
role
session
project
mission
generation
attestation reference
```

Make explicit:

> Grapher records this context but does not independently authenticate it.

This is the contract future Agent Hub work will consume.

---

89. API / LIBRARY BOUNDARY

If Grapher currently exposes only CLI commands, inspect whether the core logic can be cleanly separated into reusable application services.

Agent Hub should eventually be able to call Grapher behavior without shelling out to dozens of fragile CLI commands.

Where architecture permits, establish clean internal services for:

```
load graph
normalize graph
query graph
rank results
validate graph
mutate graph
audit graph
migrate graph
manage checkpoints
record provenance
```

Do not prematurely build a network API.

The immediate requirement is a clean callable application boundary.

---

90. DO NOT TIGHTLY COUPLE TO CURSOR

Cursor is an important consumer, not Grapher's identity.

Grapher must remain usable by:

- Cursor;
- Codex;
- command-line operators;
- future Dreadnought agents;
- future Agent Hub;
- future Auditor;
- other agnostic agents.

Cursor-specific rule generation belongs in an adapter/integration layer.

Core graph semantics must remain agent-agnostic.

---

91. DO NOT TIGHTLY COUPLE TO DREADNOUGHT

The new mission/provenance primitives are generalized infrastructure.

Do not create hard-coded classes such as:

```
DreadnoughtAcceptanceObject
CodexFieldState
AuditorSpecialRecord
```

unless those are implemented as optional profiles later.

Prefer generic concepts:

```
mission
generation
actor
role
session
handoff
acceptance
```

```
audit record
provenance
finalization
```

Dreadnought will consume these later.

---

92. NO SILENT AUTHORITY INFERENCE

This deserves repetition.

Do not infer:

```
arbiter authority
auditor authority
field ownership
```

from:

- node title;
- actor name;
- role text;
- file path;
- confident content.

At most, Grapher records:

```
declared role
```

unless an external trusted caller provides verified context.

This prevents the knowledge layer from recreating the exact case-study contamination problem internally.

---

93. NO SILENT COMPLETION INFERENCE

Do not infer completion from:

- handoff;
- process exit;
- idle state;
- agent detachment;

- report creation;
- passing tests;
- clock-out;
- acceptance by another role.

These may each be separate evidence or state records.

A future orchestration layer decides how they compose.

Grapher must preserve distinctions.

---

94. NO SILENT SCOPE COLLAPSE

Never silently merge:

```
project status
mission status
mission-generation status
task status
product readiness
```

into one generic `complete`.

Scope must remain inspectable.

---

95. NO SILENT HISTORY REWRITE

A reopened mission must create new state.

Do not overwrite the old mission's completion event and pretend it never happened.

Use:

```
new generation
new records
supersession where semantically appropriate
```

The historical record remains useful.

---

96. IMPLEMENTATION ORDER

Perform work in this order unless actual repository constraints require a documented adjustment.

Phase 1 — Reconnaissance

- architecture;
- CLI;
- persistence;
- vectors;
- Dash;
- Cursor;
- tests;
- baseline.

Do not mutate yet.

Phase 2 — Core registries

Implement:

- kinds;
- stages;
- statuses;
- workflow states;
- verification;
- node types;
- relations;
- profiles;
- aliases.

Phase 3 — Normalized model

Create unified v1/v2 reader.

Do not break v1.

Phase 4 — Schema v2 serialization

Implement:

- graph metadata;
- new node fields;
- evidence;
- scope;

- provenance;
- finalization.

Phase 5 — Mutation safety

Implement:

- atomic writing;
- backups;
- mutation journal;
- finalized-record protections.

Phase 6 — Migration

Implement v1 → v2.

Verify losslessness and idempotence before proceeding.

Phase 7 — Query and ranking

Implement:

- status awareness;
- verification awareness;
- scope;
- generation;
- provenance;
- explain ranking.

Phase 8 — Relations and supersession

Implement precise relation registry and contradiction handling.

Phase 9 — Validation and audit

Implement structural and semantic diagnostics.

Phase 10 — Checkpoints and compaction

Review-first and non-destructive.

Phase 11 — Ingest compatibility

Ensure deep ingest remains intact.

Phase 12 — Cursor integration

Generate updated rules and skills.

Phase 13 — Dash shared adapter

Eliminate duplicated graph semantics.

Phase 14 — Dash views and filtering

Including provenance/mission-history visualization.

Phase 15 — Fixtures and acceptance testing

Use:

```
CASSIO
generic non-software fixture
multi-agent case-study fixture
```

Phase 16 — Documentation

Only document behavior that actually exists.

Phase 17 — Final regression

Run all existing and new tests.

---

97. IMPLEMENTATION GATES

Do not consider a phase finished merely because code compiles.

Gate A — Compatibility

Existing v1 graph opens.

Gate B — Migration

V1 → v2 is provably lossless.

Gate C — Search

Current truth outranks stale truth appropriately.

Gate D — History

Superseded/history remains accessible.

Gate E — Multi-agent state

Mission generation and provenance can alter current-state interpretation correctly.

Gate F — Dash

Dash uses the same normalized model.

Gate G — Cursor

Generated Cursor rules match implemented semantics.

Gate H — Non-software

A non-code project works naturally.

---

98. DEFINITION OF DONE

The Grapher modernization is complete only when all of the following are true.

Generalization

Grapher can naturally represent non-software work.

Lifecycle

All six canonical stages are implemented.

Graph kinds

Kinds and lifecycle stages are independent and repeatable.

Backward compatibility

Version 1 graphs remain readable.

Migration

V1 graphs migrate losslessly and idempotently.

Truth

Current, canonical, proposed, historical, superseded, rejected, and deprecated truth are explicitly representable.

Workflow

Execution state is independent from truth status.

Verification

Verification is independent from workflow and truth.

Evidence

Verified claims can reference supporting evidence.

Provenance

Nodes can record actor/session/source context.

Provenance integrity

Declared, verified, contested, and invalidated provenance are distinguishable.

Scope

Project, mission, and mission generation can be distinguished.

History

A reopened mission does not erase an older generation.

Finalization

Official historical records cannot be casually rewritten.

Mutation journal

Semantic graph mutations can be reconstructed.

Supersession

Current replacements can point to superseded history.

Contradiction

Unresolved disagreement remains visible.

Search

Retrieval prefers appropriate current truth without erasing historical context.

Explainability

Ranking decisions can be inspected.

Checkpoints

Current-state summaries remain traceable to underlying evidence.

Audit

Graph health can be assessed without mutation.

Compaction

Sediment can be consolidated without deleting history by default.

Ingest

Documents, images, videos, and audio remain first-class semantic sources.

Cursor

Cursor searches before acting and records graph-worthy knowledge afterward.

Agent agnosticism

Core Grapher behavior does not depend on Cursor.

Agent Hub readiness

Future Agent Hub can provide actor/mission/generation context without Grapher knowing Agent Hub internals.

Auditor readiness

Future Auditor can distinguish authentic, contested, stale, and invalidated records using graph data.

Dash

Dash uses Grapher's shared normalized model.

Dash history

Dash can visualize supersession, lifecycle, and mission/provenance history.

Non-software acceptance

Generic fixture passes.

CASSIO acceptance

CASSIO migrates without semantic loss.

Control-case acceptance

The multi-agent fixture correctly expresses that technical completion may exist while authentic arbiter acceptance remains unestablished.

Tests

All existing and new tests pass.

Documentation

README and CLI help describe reality rather than intended future features.

---

99. FINAL REPORT FORMAT

When implementation is complete, return a detailed report with exactly these major sections.

1. Architecture discovered

Describe:

- language;
- CLI;
- model;
- persistence;
- vectors;
- Cursor integration;
- Dash;
- tests.

2. Baseline state

Report original test results and important original behavior.

3. Files changed

For every significant file:

```
path
purpose
major change
```

4. Schema

Show implemented v2 structure.

5. Migration

Report:

- fixture used;
- original node count;
- migrated node count;
- original edge count;
- migrated edge count before approved inference;
- semantic comparison;
- idempotence result;
- backup behavior;
- atomic-write test.

6. CLI

List actual implemented commands and flags.

Do not list theoretical commands.

7. Retrieval

Describe ranking and filtering.

Include at least one `--explain-ranking` example.

8. Provenance and mission generation

Explain the implemented model and how future Agent Hub can supply context.

9. Mutation history and finalization

Explain forensic guarantees.

10. Cursor integration

Describe generated rule/skill changes.

11. Ingest

Confirm deep media ingestion behavior remains intact.

12. Dash

Report:

- launch command;
- view modes;
- filters;
- node details;
- provenance view;
- exports;
- performance measures.

13. Tests

Provide exact test commands and results.

14. Acceptance fixtures

Report results for:

```
CASSIO
generic non-software fixture
multi-agent control-case fixture
```

15. Deliberate deviations

Identify any instruction not implemented literally and why.

16. Remaining risks

Do not hide them.

17. Handoff to Agent Hub phase

End with a compact list of interfaces now available for the next agent responsible for Agent Hub.

That handoff should answer:

> What can Agent Hub now rely on Grapher to do?

---

100. FINAL CONSTRAINT

Do not claim success because the application runs.

Do not claim success because the schema exists.

Do not claim success because tests were added.

Do not claim success because Dash displays a graph.

Do not claim success because CASSIO migrates.

This work is successful when Grapher can reliably answer the class of question that the Dreadnought system will repeatedly need to ask:

> **What is true now, within this exact scope and mission generation, according to which evidence, written by whom, with what provenance and verification, what did it replace, and what history must still be preserved to understand how we got here?**

That is the target.

Build Grapher around that question.